

hapticOS

Context you can feel

Technical companion and presentation blueprint

A proposed Android app that turns important alerts, walking directions and camera alignment into distinct vibration patterns. One cue engine coordinates output so that outdated or competing events do not create confusing guidance.

What exists today

The public mobile concept provides three clickable scenarios with scripted events and visual cue previews. It does not yet run local AI, read phone notifications, access live camera or location data, or produce native vibration. The proposed Android build is the hackathon implementation milestone.

Prototype: <https://hapticos-concept.rprashasthimmanuel.chatgpt.site>

The product insight

A vibration becomes useful when a person can recognise its meaning and trust that it is still relevant. hapticOS therefore focuses on a small learned vocabulary, current context and predictable scheduling. Rich haptic effects already exist in Android; our contribution is their coordination across selected events. [1, 2]

Submission position

Smart Living is the recommended track because the initial use cases address everyday convenience. The deck presents a bounded native build and a measurable validation plan. Gaming integrations, OEM licensing and broader accessibility applications remain future hypotheses.

Reading guide

Pages that follow explain implementation, performance, device boundaries and viability. The presentation blueprint supplies complete slide copy and visual directions. Source numbers refer to the primary references at the end. All timing figures are proposed targets. No measured performance, traction, user study or partnership is claimed.

01 Architecture and event contract

Three input paths share one output policy. Notification text may require a small local language model. Camera frames require an object detector. Route progress requires location and routing logic. A language model does not calculate walking directions or render individual vibration pulses.

Layer	Proposed implementation	Purpose
Native application	Kotlin, Jetpack Compose, coroutines	Interface, onboarding and explicit mode control
Input adapters	NotificationListenerService; one supported cellular call path; CameraX; location and route provider	Normalise permitted inputs
Local interpretation	Qwen2.5 0.5B Instruct via llama.cpp; MediaPipe EfficientDet Lite0 candidates	Classify selected short text and detect supported objects
Shared cue engine	Typed events, bounded scheduler, effect capability map	Validate, prioritise, deduplicate and cancel
Output	VibratorManager and VibrationEffect	Request supported effects through Android

Event contract

Each event carries a kind, source, monotonic timestamp, expiry, source sequence, route or target identity, and deduplication key. Keeping identity and freshness explicit prevents an old result from becoming a new instruction. Model output maps into a fixed set of permitted labels; it cannot create arbitrary output commands.

Scheduling rules

Use a single owner for hapticOS output. Discard expired events before arbitration and again before dispatch. Coalesce successive camera updates into the latest target state. Deduplicate notification updates. When an interrupt ends, obtain current route or target state rather than replaying queued history.

Stop obsolete guidance when location or target state becomes invalid. Resolve remaining competing cues with documented priorities, user preferences and cooldowns. The scheduler manages hapticOS cues only; other apps and Android policies remain outside its global control.

Example: a call interrupts a pending turn. When the call cue ends, the old turn has expired. The engine waits for fresh route progress instead of replaying that turn.

Implementation references: [2–9]. The model and detector are candidates, not verified deployment results.

02 Three constrained implementation paths

Alerts and supported calls

Ask the user to enable notification access and select permitted sources. Apply explicit rules first for configured events. For eligible short text, request a fixed output category from a compact local model, validate the response and enforce a deadline. If access is denied, content is redacted or inference times out, use only an authorised rule fallback or produce no new cue. A late response must not trigger a second vibration. [5, 7, 9, 18]

A proposed cellular call path uses TelephonyCallback.CallStateListener with READ_PHONE_STATE. It reports call state, not caller intent or urgency. Coverage of third party calling apps is not assumed. Users may need to adjust source notification vibration settings to avoid duplicate cues. [6]

Walking guidance

A walking route provider supplies instructions and Android location updates advance route progress. Assign distinct learned rhythms to left, right and arrival. Include a route identifier, instruction sequence and expiry in each event. Clear obsolete cues when the route changes, location becomes unreliable or the user ends navigation. Route service connectivity, terms and cost must be checked during integration.

This is guidance for a selected walking route. It is not obstacle detection or a substitute for established mobility aids. The current concept shows a right turn followed by a left turn; the live event build must prove alignment between route data and cues.

Camera finder

Enable CameraX only in finder mode. Use KEEP_ONLY_LATEST so analysis does not accumulate old frames. Begin with bottle and two other categories supported by the detector. Use target identity, a short confirmation window and hysteresis to reduce flicker before mapping horizontal alignment to a cue. Stop guidance on target loss. [4, 8]

A starting analysis cadence of 5 to 10 frames per second is a tuning candidate. A bounding box can indicate alignment in the image; it cannot establish reliable distance or a safe path. The app cannot locate a bottle hidden under a bed when the camera cannot see it.

Learning the cues

Provide a short tutorial with visual descriptions and replay controls. Test whether the mode context sufficiently distinguishes rhythms. Allow a supported alternative pattern where recognition is poor. The visual prototype does not establish tactile learnability.

03 Performance, resource use and privacy

Proposed target	Measurement boundary	Failure behaviour
p95 below 50 ms	Rule event callback to Android vibration request	Log missed target; inspect scheduling delay
p95 below 300 ms	Analysed camera frame timestamp to output request, after acquisition	Drop stale frames and obsolete target state
1.5 second deadline	Eligible text classification request to validated result	Use explicit rule fallback; discard late result

These targets are acceptance goals, not achieved results or universal human perception thresholds. The camera trace must include confirmation delay. Separate cold model loading, warm inference, scheduling and API dispatch. Report p50 and p95 with sample counts and exact phone, OS and runtime configuration.

Physical onset is a separate measurement

The vibration request returning does not prove that the actuator has started. Android traces and monotonic app timestamps measure software stages. Credible physical onset measurements require external instrumentation or a suitably validated measurement setup. Motor dynamics and manufacturer implementation affect the physical result. [3, 10]

Battery strategy and test

Analyse only active finder sessions, skip unnecessary text inference, limit context and output length, reuse supported effect definitions and enforce cooldowns. Benchmark CPU inference first; adopt acceleration only after verifying model support, thermal behaviour and an actual benefit on the event phone.

Compare identical scripted sessions under matched brightness, network, duration and device conditions. Compare haptics enabled versus disabled to estimate cue cost. Compare continuous analysis versus bounded analysis to test the sensing policy. Use supported Android power profiling or external measurement. Do not infer precise savings from a short battery percentage change. [16]

Data handling

Process selected text and frames locally in the proposed design. Do not retain message bodies or camera frames by default. Record event categories and timing for diagnostics with clear user control. Local inference does not make the entire application offline: route services may still need connectivity. Device access must remain revocable.

04 Android boundaries and iQOO fit

hapticOS is an application and orchestration layer. It is not a replacement operating system, kernel driver or manufacturer haptic stack.

Boundary	What the application can claim
App and framework	Request supported VibrationEffect patterns through Android APIs; query capabilities and choose fallbacks
System service	Android enforces lifecycle, usage and system policies
Vibrator HAL and driver	The manufacturer maps requests to motor behaviour and calibration
Electrical actuation	Raw PWM, driver signals and arbitrary motor axes are not ordinary app controls

HAL means hardware abstraction layer. App waveform timings and amplitudes are not raw pulse width modulation commands. The actual electrical drive design is device dependent. Unsupported primitive compositions require a supported fallback. Capability dependent envelope APIs are optional, not a dependency for this event build. [2, 3, 10, 17]

Permissions and lifecycle

Notification listening requires special user access, and Android may redact sensitive content. Camera and location require their respective permissions. Ongoing background work must follow applicable foreground service and location rules. Supply accurate vibration usage attributes; do not treat them as a way around Do Not Disturb. [3, 5, 6, 18]

Specific ecosystem connection

The official iQOO 15 page describes Snapdragon 8 Elite Gen 5, OriginOS 6, a single motor integrating X and Z axes, and Office Kit. These demonstrate relevant ecosystem capabilities. They do not identify the loaner phone or establish independent axis control through public app APIs. Verify the actual device before selecting effects or acceleration. [11, 12]

Office Kit supports the phone driven development workflow during the event and file or screen bridging when permitted. It is a build tool, not a required service for an eventual hapticOS user. The camera, location, local computation and vibration hardware make the phone essential to the actual product.

Competition alignment

The event guide allocates 30% to product quality, 20% to novelty and impact, 15% to creative phone use, 15% to technical depth, 10% to Office Kit and 10% to presentation. These are event judging weights, not confirmed idea screening weights. The event’s originality and device usage rules still apply. [15]

05 Viability, delivery and decisive evidence

A narrow audience and a testable reason to return

Start with Android users who want fewer screen checks during everyday tasks. Test whether they can learn the cue vocabulary and choose to keep using at least one mode. Novelty rests in consistent coordination across selected events. The individual features already have alternatives, including Android effects, Apple Watch turn taps and Seeing AI object finding. [1, 13, 14]

Business path

Begin with a small Android beta to test useful repeated behaviour. If the scheduler, capability mapping and learning tools prove reusable, explore developer or OEM licensing and integration support. This is a business hypothesis, not a partnership or revenue claim. Device coverage, onboarding permissions, route service fees, support and distribution are material costs. No defensible market size can be claimed without a defined buyer and evidence of demand.

Gaming would require game integration and its own timing requirements. Accessibility expansion requires research with intended users and should supplement established aids. Neither is part of the core event promise.

Build scope for three people

Target selected notifications, one supported call path, one walking route and three detector categories. Proposed ownership: app and haptics; models and camera; route integration and validation. Team credentials are not asserted because verified biographies and achievements have not been supplied.

First establish native cues, permissions and the event contract. Then connect one input path at a time. Reserve the final window for failed permissions, target loss, stale routes, cold inference and competing cues. Thirty elapsed event hours include evaluation and other activities. Follow the actual Green Light and Red Light schedule. [15]

The first minute of the demonstration

Show a promotion and a ride arrival event producing different attention decisions. Trigger a supported call while a turn is pending, then show the expired turn being discarded. Aim at a visible bottle, move it out of view and show guidance stopping. Label simulated inputs whenever live integration is unavailable. In the current browser prototype, all inputs and vibration previews are simulated.

Evidence that changes the decision

Record recognition errors after a short tutorial, completion of sample tasks, software latency distributions and failure handling. Report actual sample sizes and conditions. A small convenience test does not establish accessibility effectiveness. The strongest next milestone is a reliable native loop on the event phone, not an expanded feature list.

Presentation blueprint

01 hapticOS

Context you can feel

- Recognise important alerts through distinct vibration patterns.
- Follow walking directions and align the camera with visible objects.
- Keep timing predictable with local inference and one cue engine.

Visual direction. Place the editable product name and definition on the left. Use the conceptual phone image on the right. Label the stage as an idea submission with a clickable concept prototype.

02 The problem is context and coordination

- An alert can signal activity without explaining whether it needs attention.
- Directions and camera guidance often require another screen check.
- Continuous sensing and repeated cues can add unnecessary work and distraction.

Visual direction. Use three aligned text sections for attention, context and resource cost. Add a short factual note below: Android already supports rich effects. Keep the problem distinct from an unsupported claim that Android APIs are slow or inefficient.

Source support: [1], [2].

Presentation blueprint

03 Three modes with one learned vocabulary

- Alerts: distinguish a time sensitive message from a supported incoming call.
- Navigate: learn separate rhythms for left turns, right turns and arrival.
- Find: guide camera alignment toward a visible object and signal target loss.

Visual direction. Show the three use cases as three horizontal rows. Pair each with one concrete example: ride arrival, a walk to a café and a visible bottle. Avoid presenting gaming or assistive navigation as part of the event scope.

04 Architecture built around a single cue engine

- Adapters convert permitted phone inputs into typed events.
- Local models interpret text or frames only where needed.
- The engine validates freshness and priority before requesting a supported effect.

Visual direction. Use an editable system diagram. Put three input branches across the top: notification text, route events and camera frames. Join them at a typed event stream, then the cue engine, then Android APIs. Draw a clearly labelled boundary above the manufacturer HAL and motor.

Source support: [2], [3], [4], [5], [6], [7], [8], [9].

Presentation blueprint

05 Fresh cues win over queued history

- Reject expired events and superseded route instructions.
- Deduplicate repeated detections and coalesce camera updates.
- Resolve competing hapticOS cues with explicit rules and a bounded queue.
- Cancel obsolete output and recheck the current state before resuming.

Visual direction. Show an editable event flow with validity checks, arbitration and effect dispatch. Include a small example trace: a turn is pending, a call interrupts, the old turn expires, fresh route data supplies the next cue.

Source support: [2], [3].

06 Performance is a testable contract

- Target p95 under 50 ms from a rule event callback to the Android vibration request.
- Target p95 under 300 ms from an analysed camera frame to the output request.
- Apply a 1.5 second deadline to text classification, then use explicit fallback rules.
- Report inference and physical actuator onset separately from software dispatch.

Visual direction. Use three large numeric targets with their exact measurement boundaries below. Label every number Proposed target, not measured. Place a compact timing strip showing capture, inference, arbitration, API request and physical onset.

Source support: [3], [10].

Presentation blueprint

07 Built for the iQOO phone

- Use the camera, location and vibrator as essential product inputs and output.
- Query effect support on the loaner and select tested fallbacks.
- Keep camera analysis active only in finder mode and limit repeated inference.
- Use Office Kit throughout the phone driven development workflow.

Visual direction. Place the phone's role on the left and compute and development decisions on the right. Include a small factual note: iQOO 15 advertises a single X and Z axis motor, but the loaner and public axis control are unconfirmed.

Source support: [2], [4], [11], [12].

08 A focused start and a credible growth path

- Start with Android users seeking fewer screen checks during everyday tasks.
- Test repeated use and pattern recognition before expanding the feature set.
- Explore developer or OEM licensing only after the engine proves useful.
- Treat gaming and accessibility extensions as future validation tracks.

Visual direction. Use a flat progression from consumer concept to tested engine to partner integration. Below it, compare existing alternatives with the narrower contribution of hapticOS. Avoid market size graphics without a sourced market definition.

Source support: [1], [13], [14].

Presentation blueprint

09 A 30 hour scope the team can execute

- Build selected notification support, one call path, one route and three object categories.
- Assign ownership across app and haptics, models and camera, and route integration.
- Validate the smallest complete path before adding optional hardware acceleration.
- Reserve the final build window for failure tests and the phone demo.

Visual direction. Use a three phase timeline: foundation, integration and validation. Place the three proposed workstreams beneath it. Make the scope count prominent and describe phases rather than inventing the organiser's exact schedule.

Source support: [15].

10 Evidence the live demo must produce

- Show cue recognition after a short tutorial and record confusions.
- Measure task completion and software timing on the actual iQOO phone.
- Replay competing events and confirm that stale output stops.
- Compare matched workloads for energy cost before claiming battery savings.

Visual direction. Use a compact evidence matrix with four rows: learnability, latency, robustness and energy. Show measurement method and the decision it informs. Label the matrix Planned validation and leave out invented results.

Source support: [16].

Presentation blueprint

11 The mobile concept is ready to explore

- A public link opens the phone interface without a ChatGPT login.
- Three clickable scenarios show alerts, a walking route and camera alignment.
- Visual cue previews communicate the interaction before native implementation.
- The event build will add real inputs, local inference and physical vibration.

Visual direction. Show three editable mobile wireframes with the current screen labels. Caption them Interface schematics based on the prototype. Put the public prototype link beneath the screens. Actual screenshots can replace the schematics without changing the slide story.

12 Appendix Android and hardware boundaries

- The app requests VibrationEffect output through Android services.
- The manufacturer's HAL and driver control motor actuation and calibration.
- PWM and raw driver signals are outside the ordinary app contract.
- Permission, background and system vibration policies still apply.

Visual direction. Use a vertical boundary diagram from hapticOS to Android APIs, system service, vibrator HAL, driver and actuator. Mark the top layers App control and the lower layers Manufacturer control. Add concise permission notes beside the diagram.

Source support: [2], [3], [5], [6], [10], [17], [18].

Presentation blueprint

13 Primary sources and claim boundaries

- Android and AOSP documentation define the supported API and hardware boundaries.
- iQOO's product page supports ecosystem facts, not loaner or performance assumptions.
- Official product documentation establishes existing alternatives.
- All hapticOS latency, usability and business outcomes remain proposed or unmeasured.

Visual direction. Use a readable reference page grouped by Android, local AI, iQOO and alternatives. Put complete URLs and source to claim mapping in the companion document and speaker notes.

Source support: [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18].

Primary references

Primary documentation reviewed for this submission. Source support concerns platform capabilities and product comparisons, not measured hapticOS outcomes. Research date: 22 September 2026.

[1] Android haptics design principles

<https://developer.android.com/develop/ui/views/haptics/haptics-principles>

[2] Android haptics API reference

<https://developer.android.com/develop/ui/views/haptics/haptics-apis>

[3] VibratorManager API

<https://developer.android.com/reference/android/os/VibratorManager>

[4] CameraX image analysis

<https://developer.android.com/media/camera/camerax/analyze>

[5] NotificationListenerService API

<https://developer.android.com/reference/android/service/notification/NotificationListenerService>

[6] CallStateListener API

<https://developer.android.com/reference/android/telephony/TelephonyCallback.CallStateListener.html>

[7] Qwen2.5 0.5B Instruct model card

<https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct>

[8] MediaPipe object detector

https://developers.google.com/edge/mediapipe/solutions/vision/object_detector

[9] llama.cpp local inference runtime

<https://github.com/ggml-org/llama.cpp>

Primary references

Primary documentation reviewed for this submission. Source support concerns platform capabilities and product comparisons, not measured hapticOS outcomes. Research date: 22 September 2026.

[10] Android actuator behaviour

<https://developer.android.com/develop/ui/views/haptics/actuators>

[11] Official iQOO 15 product page

<https://www.iqoo.com/in/products/iqoo15>

[12] Qualcomm QNN execution provider

<https://www.qualcomm.com/developer/blog/2026/05/qualcomm-launches-the-first-onnx-runtime-plugin-execution-provider>

[13] Apple Watch Maps directions

<https://support.apple.com/en-ca/guide/watch/apdea7480950/watchos>

[14] Microsoft Research Find My Things

<https://www.microsoft.com/en-us/research/story/find-my-things/>

[15] iQOO City Battles guide and rules

<https://iqoo.reskilll.com/guide>

[16] Android battery profiling

<https://developer.android.com/topic/performance/power/setup-battery-historian>

[17] AOSP haptics implementation

<https://source.android.com/docs/core/interaction/haptics/haptics-implement>

[18] Android 15 sensitive notifications

<https://developer.android.com/about/versions/15/behavior-changes-all#otp-redaction>